1. What is a Structure in C#?

A **Structure (struct)** is a **value type** used to represent **lightweight objects**.

It is similar to a **class**, but with key differences:

- Stored in **stack memory** (usually)
- **Value type**
- Used for **small data containers**
- **Copied by value**
- Cannot support inheritance from other classes

Common examples from .NET:

- DateTime
- TimeSpan
- Guid
- Point

These are implemented as **structs** because they represent **simple data values**.

---

2. Syntax of a Structure

```
struct StructName
{
    // Fields
    // Constructors
    // Methods
    // Properties
}
```

Example:

```
struct Point
```

```
{
    public int X;
    public int Y;
}
```

Usage:

```
Point p1;
p1.X = 10;
p1.Y = 20;

Console.WriteLine($"X: {p1.X}, Y: {p1.Y}");
```

Output

```
X: 10, Y: 20
```

---

3. Structure Example (Real World)

Example: **Employee Salary Structure**

```
struct Employee
{
    public int Id;
    public string Name;
    public double Salary;

    public void Display()
    {
        Console.WriteLine($"Id: {Id}");
        Console.WriteLine($"Name: {Name}");
        Console.WriteLine($"Salary: {Salary}");
```

```
    }
}
```

Program

```
class Program
{
    static void Main()
    {
        Employee emp;

        emp.Id = 101;
        emp.Name = "Ravi";
        emp.Salary = 50000;

        emp.Display();
    }
}
```

Output

Id: 101

Name: Ravi

Salary: 50000

---

4. Structure Characteristics

Important properties of structs:

Feature	Struct
Type	Value Type
Memory	Stack
Inheritance	Not Supported
Constructors	Supported
Default Constructor	Always exists
Destructor	Not allowed
Polymorphism	Limited

---

5. Value Type Behavior (Most Important Concept)

Structs are **copied by value**, unlike classes.

Example:

```
struct Point
{
    public int X;
    public int Y;
}


class Program
{
    static void Main()
    {
        Point p1;
        p1.X = 10;
```

```
        p1.Y = 20;

        Point p2 = p1;

        p2.X = 100;

        Console.WriteLine(p1.X);
        Console.WriteLine(p2.X);
    }
}
```

Output

10

100

Why?

Because **p1 and p2 are separate copies**.

Memory visualization:

Stack

p1 -> X=10 Y=20

p2 -> X=100 Y=20

---

6. Struct with Constructor

Structs can have **parameterized constructors**.

```
struct Rectangle
{
    public int Length;
```

```
    public int Width;

    public Rectangle(int l, int w)
    {
        Length = l;
        Width = w;
    }

    public int Area()
    {
        return Length * Width;
    }
}
```

Usage

```
Rectangle r = new Rectangle(10,5);

Console.WriteLine(r.Area());
```

Output

50

---

7. Struct with Properties

```
struct Product
{
    public int Id { get; set; }
    public string Name { get; set; }
```

```
    public void Display()
    {
        Console.WriteLine($"{Id} {Name}");
    }
}
```

Usage

```
Product p = new Product();

p.Id = 1;
p.Name = "Laptop";

p.Display();
```

---

8. Struct with Methods

```
struct Calculator
{
    public int Add(int a, int b)
    {
        return a + b;
    }

    public int Multiply(int a, int b)
    {
        return a * b;
    }
}
```

Usage

```
Calculator calc = new Calculator();


Console.WriteLine(calc.Add(10,20));

Console.WriteLine(calc.Multiply(5,6));
```

---

9. Struct vs Class (Very Important Interview Question)

Feature	**Struct**	**Class**
Type	Value Type	Reference Type
Memory	Stack	Heap
Inheritance	Not supported	Supported
Default Constructor	Always present	Optional
Copy Behavior	Value copy	Reference copy
Performance	Faster for small data	Good for large objects

---

10. When Should We Use Struct?

Use **struct** when:

✅ Object is **small**
✅ Represents **single value**
✅ Immutable data
✅ High performance needed

Examples

- Coordinates
- Color
- Date

- Money
- Measurement

Example

```
struct Coordinate
{
   public int X;
   public int Y;
}
```

---

11. Immutable Struct (Best Practice)

Most structs should be **immutable**.

Example:

```
readonly struct Point
{
   public int X { get; }
   public int Y { get; }

   public Point(int x, int y)
   {
      X = x;
      Y = y;
   }
}
```

Usage

```
Point p = new Point(10,20);
```

Now values **cannot change**.

---

12. Modern C# Struct Features (C# 10+ / C# 14)

Modern C# introduced improvements:

1️⃣ **readonly struct**

Prevents modification.

```
readonly struct Temperature
{
    public double Value { get; }

    public Temperature(double value)
    {
        Value = value;
    }
}
```

2️⃣ **record struct**

Used for **data structures with value equality**

```
public record struct Student(int Id, string Name);
```

Usage

```
Student s1 = new Student(1,"John");
Student s2 = new Student(1,"John");

Console.WriteLine(s1 == s2);
```

Output

```
True
```

3️⃣ ref struct

Used for **stack-only structures**

Example: Span<T>

```
ref struct MyStruct
{
    public int Value;
}
```

Restrictions

- Cannot be boxed
- Cannot be stored in heap
- Cannot implement interfaces

---

14. Limitations of Struct

Struct **cannot**

❌ inherit from another struct/class
❌ define destructor
❌ support parameterless constructor (before C# 10)
❌ have protected members

But they **can**

✓ implement interfaces
✓ contain methods
✓ contain properties
✓ contain constructors

---

17. Real .NET Struct Examples

.NET uses structs widely:

DateTime

TimeSpan

Guid

Decimal

Point

Span<T>

Example:

```
DateTime today = DateTime.Now;


Console.WriteLine(today);
```

💡 **Simple Rule**

Use:

Class → Complex objects

Struct → Small data objects